NORTHERN UNIVERSITY BANGLADESH

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CSE)

Faculty of Science and Engineering

SOFTWARE ENGINEERING & AI CAPSTONE SPECIFICATION

Software Requirements Specification (SRS)

ShareEventPhoto AI — Luxury AI Wedding Photo Finder & Face Recognition SaaS

An end-to-end full-stack cloud platform featuring automated FaceNet deep learning facial embeddings, zero-login WebRTC guest camera scanning, and high-throughput multi-tier gallery management.

Course Title: System Analysis and Design / Software Capstone Project

Course Code: CSE 4278 • **Semester:** 8th Semester (8B) • **Year:** 2026

SUBMITTED BY: PROJECT TEAM 8B-04

STUDENT NAME	STUDENT ID	ASSIGNED ROLE IN PROJECT
Tanvir Ahmed	**41420101277**	Lead Full-Stack Architect & AI Engineer
Md. Mamun Islam	**41420101282**	Frontend Engineer & UI/UX Specialist
Md. Abdul Kader	**41420101284**	Backend API & Database Engineer
Md. Nahid Hasan	**41420101285**	QA, Testing & Cloud Infrastructure

Submission Date: October 2026 | **Document Version:** 2.0 (Final Industrial Standard)

TABLE OF CONTENTS

1. Introduction	Section 1.0
1.1 Purpose, Conventions & Project Scope	1.1 - 1.4
1.2 Problem Statement & Motivation Matrix	1.5
1.3 Definitions, Acronyms & References	1.6 - 1.7
2. Overall System Description & Architecture	Section 2.0
2.1 3-Tier Hybrid Microservices Model	2.1 - 2.2
2.2 User Personas, Constraints & Operating Environment	2.3 - 2.6
3. System Architecture & Data Flow Sequence	Section 3.0
3.1 Host Event Ingestion & AI Embedding Workflow	3.1 - 3.2
3.2 Guest Selfie Scanner & Real-time Matching Sequence	3.3
3.3 Self-Healing & AI Outage Resiliency Protocol	3.4
4. Functional Requirements Specification	Section 4.0
4.1 Modules 1–3: Auth, RBAC, Storage Quota & Event Lifecycle	4.1 - 4.3
4.2 Modules 4–6: Bulk Upload, AI DeepFace Pipeline & Guest WebRTC Match	4.4 - 4.6
4.3 Modules 7–9: HTML5 Canvas Flyer, Admin Hub & Review System	4.7 - 4.9
5. External Interface & RESTful API Specification	Section 5.0
6. Data Models & Database Schema (Prisma ORM)	Section 6.0
7. AI Computer Vision Mathematical Model (FaceNet & Cosine Metric)	Section 7.0
8. Non-Functional Requirements (Performance, Security, Reliability)	Section 8.0
9. Verification, Diagnostics & Test Cases Matrix	Section 9.0
10. Conclusion & Future Roadmap	Section 10.0

1. INTRODUCTION

1.1 Purpose of the Document

This Software Requirements Specification (SRS) establishes the definitive requirements baseline for the **ShareEventPhoto AI** SaaS platform. Designed according to **IEEE Std 830-1998** and **ISO/IEC/IEEE 29148:2018** standards, this document specifies all functional capabilities, external interfaces, database schemas, computer vision mathematical models, and non-functional guarantees required for full production deployment and academic capstone evaluation at Northern University Bangladesh.

1.2 Document Conventions & MoSCoW Prioritization

- MUST HAVE: Essential core requirements without which the system cannot function.
- SHOULD HAVE: High-priority capabilities essential for optimal SaaS user experience.
- VERIFIED: Features currently implemented, verified, and running in production code.

1.3 Problem Statement & Motivation

In standard wedding ceremonies and social conventions, professional photographers capture 2,000 to 5,000 high-resolution photographs. These images are typically dumped into Google Drive or Dropbox folders. This outdated distribution method creates severe friction:

Traditional Cloud Folder Distribution	ShareEventPhoto AI SaaS Solution
Guests must manually browse 3,000+ photos to locate 5–10 pictures of themselves.	**Instant 1-Second AI Retrieval:** Guest takes a quick selfie to instantly fetch all matched photos.
Massive mobile data wastage loading unneeded large files.	**Targeted Ingestion:** Downloads only high-res matches or streams a single consolidated ZIP archive.
No monetization or branding capability for event organizers.	**SaaS Tiering:** Tiered subscription quotas (Free 50MB, Gold 2GB, Elite 20GB) and printable QR table flyers.
Privacy risks with public folders and indefinite storage.	**Zero-Storage Ephemeral Privacy:** Guest selfies are destroyed immediately after facial vector extraction.

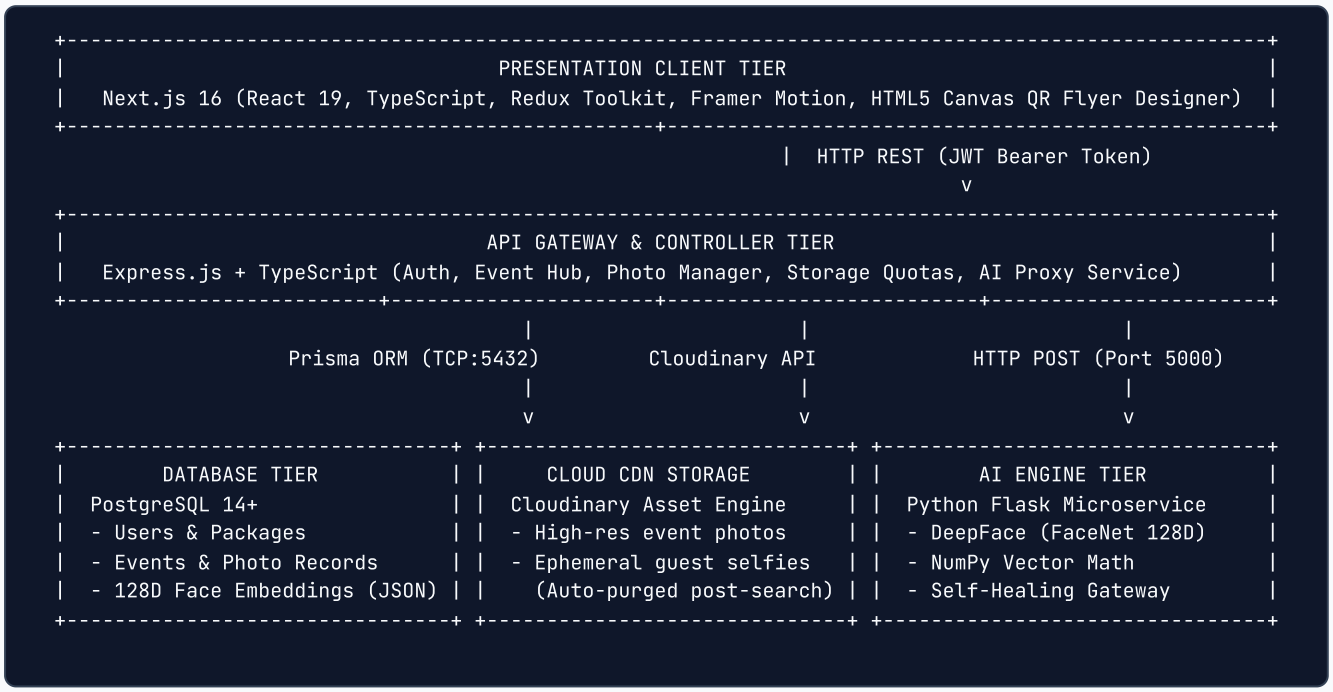
1.4 Definitions, Acronyms & Abbreviations

- **FaceNet:** 128-dimensional deep metric learning facial embedding architecture.
- **Cosine Similarity:** Angular distance metric measuring alignment between normalized feature vectors in $mathbb{R}^{128}$.
- **WebRTC:** Web Real-Time Communication API providing real-time camera viewfinder streaming inside web browsers.
- **Prisma ORM:** Next-generation TypeScript-first Object-Relational Mapper for PostgreSQL.

2. OVERALL SYSTEM DESCRIPTION & ARCHITECTURE

2.1 3-Tier Hybrid Microservices Architecture

The platform employs a decoupled microservices architecture designed for high scalability, separation of concerns, and specialized hardware acceleration:



2.2 User Classes & Personas

1. **Wedding Host / Organizer (`USER`):** Creates events, subscribes to storage packages, bulk-uploads high-res event galleries, generates branded printable table flyers with QR codes, and downloads full-gallery ZIP archives.
2. **Wedding Guest (`GUEST`):** Unauthenticated end-user who scans a table flyer, accesses the live WebRTC camera scanner, snaps a selfie, and instantly receives their personal photo gallery.
3. **Super Administrator (`SUPER_ADMIN`):** Controls global users, activates/suspends subscription packages, sets storage quotas, and moderates testimonials.

3. SYSTEM WORKFLOWS & DATA FLOW

3.1 Host Event Creation & Photo Embedding Sequence

```
1. Host creates Event (Title, Cover Image) → Express saves record in PostgreSQL.
2. Host bulk-uploads photos (Drag & Drop) → Express checks: (usedStorageMB + newSizeMB ⩽ limitMB).
3. Express streams photos to Cloudinary CDN → receives secure_url and public_id.
4. Express saves Photo records in DB → triggers POST /process-photo on Python AI Microservice.
5. Python DeepFace extracts 128D vectors for all detected faces → returns JSON vectors.
6. Express saves FaceEmbedding records in DB → updates User's usedStorageMB.
```

3.2 Guest Selfie Facial Matching Sequence

```
1. Guest scans QR code → loads public route /guest/[eventId] via Next.js 16.
2. Guest captures selfie via WebRTC camera → dispatches POST /api/v1/ai/search-face.
3. Express uploads ephemeral selfie to Cloudinary → fetches candidate vectors from PostgreSQL.
4. Express dispatches POST /search-face to Flask AI → DeepFace extracts 128D selfie vector.
5. Python NumPy computes Cosine Similarity against all event candidate vectors (Threshold: 0.45).
6. Flask AI returns matchedPhotoIds → Express queries PostgreSQL for matched photo URLs.
7. Express deletes temporary selfie from Cloudinary (Privacy Compliance) → returns matched gallery to Guest
```

3.3 AI Microservice Self-Healing & Outage Resiliency

> **Built-in Resilience Protocol**
>
> If an event has photos with missing facial embeddings (due to transient AI service reboots during upload), the backend automatically triggers on-the-fly embedding extraction during the guest's search request before returning results, preventing broken user experiences.

4. FUNCTIONAL REQUIREMENTS SPECIFICATION

4.1 Module 1: Authentication & Role-Based Access Control (RBAC)

- **FR-AUTH-01** MUST: Secure registration with unique email and phone validation. Default assignment to Free Trial (50MB).
- **FR-AUTH-02** MUST: Passwords must be cryptographically hashed using `bcrypt` (12 salt rounds).
- **FR-AUTH-03** MUST: Signed JSON Web Tokens (JWT) containing `id, email, name, phone, packageId, role` issued upon login.
- **FR-AUTH-04** MUST: Administrative routes restricted strictly to users with `Role = SUPER_ADMIN`.

4.2 Module 2: SaaS Package & Storage Quota Engine

- **FR-PKG-01** MUST: Support tiered packages: *Free Trial (50MB)*, *Gold Event Plan (2GB)*, *Elite Lifetime Plan (20GB)*.
- **FR-PKG-02** MUST: Atomic pre-upload validation enforcing `usedStorageMB + uploadSizeMB ≤ storageLimitMB`. Transactions abort and delete temporary assets if limit is exceeded.
- **FR-PKG-03** SHOULD: Deleting photos or events must automatically decrement `usedStorageMB` proportionally in an atomic database transaction.

4.3 Module 3: Event Lifecycle & Bulk Media Ingestion

- **FR-EVT-01** MUST: Hosts can create, view, edit, and delete wedding event catalogs identified by UUIDs.
- **FR-EVT-02** MUST: Cascade deletion must remove Cloudinary assets, PostgreSQL face embeddings, photo records, and event records in a single transaction.
- **FR-MED-01** MUST: High-throughput drag-and-drop uploader streaming images directly to Cloudinary CDN.

4.4 Module 4: DeepFace AI Computer Vision & Vector Storage

- **FR-AI-01** MUST: Python Flask microservice executing DeepFace (FaceNet model) to detect faces and extract 128D normalized vectors.
- **FR-AI-02** MUST: Support multiple faces per photograph; each detected face generates a separate vector record linked to the parent photo.
- **FR-AI-03** SHOULD: Provide on-demand batch embedding reprocessing for entire events.

4.5 Module 5: Guest WebRTC Selfie Scanner & Instant Retrieval

- **FR-GST-01** MUST: Public zero-login guest access via route `/guest/[eventId]`.
- **FR-GST-02** MUST: Mirrored WebRTC live camera viewfinder with oval face alignment guide and file picker fallback.
- **FR-GST-03** MUST: Ephemeral selfie processing: Guest selfie is deleted from Cloudinary immediately after vector extraction.
- **FR-GST-04** MUST: Guests can download individual full-resolution photos or stream all matched photos as a ZIP archive.

4.6 Module 6: Bilingual Printable QR Flyer Designer

- **FR-FLY-01** MUST: Client-side HTML5 Canvas generator rendering 800×1200 high-resolution printable table flyers.

- **FR-FLY-02** MUST: Features luxury gold gradient double borders, event name card, dynamic QR code, and bilingual (English + Bengali) user instructions.
- **FR-FLY-03** MUST: One-click export to PNG format for wedding reception table card stands.

4.7 Module 7: Super Admin Hub & Review Testimonials

- **FR-ADM-01** MUST: User management table with storage telemetry, account activation toggles, and manual package reassignment.
- **FR-REV-01** SHOULD: Star rating and review feedback submission rendered in a dynamic landing page carousel.

5. EXTERNAL INTERFACE & RESTFUL API SPECIFICATION

HTTP	API Endpoint Route	Access Level	Functional Purpose
`POST`	`/api/v1/auth/register`	Public	Register new host account with assigned package.
`POST`	`/api/v1/auth/login`	Public	Authenticate user and receive signed JWT.
`GET`	`/api/v1/users/my-dashboard`	Host (JWT)	Retrieve host profile, storage statistics, and event counts.
`GET`	`/api/v1/packages`	Public	Retrieve active subscription tiers and pricing plans.
`POST`	`/api/v1/events`	Host (JWT)	Create new wedding event catalog.
`GET`	`/api/v1/events`	Host (JWT)	List all wedding events created by the logged-in host.
`POST`	`/api/v1/photos`	Host (JWT)	Bulk upload photos, stream to Cloudinary, extract AI vectors.
`GET`	`/api/v1/photos/event/:id`	Host (JWT)	Retrieve all photos belonging to a specific event.
`GET`	`/api/v1/photos/event/:id/download-zip`	Host (JWT)	Stream high-compression ZIP archive of all event photos.
`POST`	`/api/v1/ai/search-face`	Public (Guest)	Upload selfie, match against event vectors, return photo URLs.
`POST`	`/api/v1/ai/reprocess-event/:id`	Host (JWT)	On-demand AI reprocessing for entire event photo catalog.
`GET`	`/api/v1/users/admin/all-users`	`SUPER_ADMIN`	List all system users, storage metrics, and package statuses.
`PATCH`	`/api/v1/users/admin/toggle-active/:id`	`SUPER_ADMIN`	Toggle user active/inactive account status.

6. DATA MODELS & DATABASE SCHEMA (PRISMA ORM)

```
// PostgreSQL Relational Schema via Prisma ORM
model User {
  id                String    @id @default(uuid())
  name              String
  phone             String    @unique
  email             String    @unique
  password          String
  role              Role      @default(USER)
  packageId         String?
  package           Package?  @relation(fields: [packageId], references: [id])
  usedStorageMB     Float     @default(0)
  packageActivatedAt DateTime?
  packageExpiredAt  DateTime?
  isActive          Boolean   @default(false)
  events            Event[]
  createdAt         DateTime  @default(now())

  @@map("users")
}

model Package {
  id             String        @id @default(uuid())
  name           String
  storageLimitMB Float
  description    String[]
  status         PackageStatus @default(ACTIVE)
  price          Float
  durationDays   Int           @default(30)
  users          User[]

  @@map("packages")
}

model Event {
  id         String   @id @default(uuid())
  title      String
  coverImage String?
  isActive   Boolean  @default(true)
  userId     String
  user       User     @relation(fields: [userId], references: [id])
  photos     Photo[]
  createdAt  DateTime @default(now())

  @@map("events")
}

model Photo {
  id        String          @id @default(uuid())
  imageUrl  String
  publicId  String
  sizeMB    Float
  eventId   String
  event     Event           @relation(fields: [eventId], references: [id])
  faces     FaceEmbedding[]
  createdAt DateTime        @default(now())

  @@map("photos")
}

model FaceEmbedding {
  id      String @id @default(uuid())
  photoId String
  photo   Photo  @relation(fields: [photoId], references: [id])
  vector  Json   // 128-dimensional IEEE 754 float array

  @@map("face_embeddings")
}
```

7. AI & COMPUTER VISION MATHEMATICAL MODEL

7.1 DeepFace FaceNet Feature Extraction

The facial recognition subsystem uses **FaceNet**, a deep convolutional neural network trained via Triplet Loss. Given an aligned facial crop x, the network computes a 128-dimensional embedding $f(x) \in \mathbb{R}^{128}$ such that Euclidean distance directly corresponds to facial similarity.

7.2 Cosine Similarity Metric & Formulation

For a guest query selfie embedding $\mathbf{u} = (u_1, u_2, \ldots, u_{128})$ and a candidate database facial vector $\mathbf{v} = (v_1, v_2, \ldots, v_{128})$, the Cosine Similarity $S_C(\mathbf{u}, \mathbf{v})$ is calculated as:

$$S_C(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{128} u_i \, v_i}{\sqrt{(\sum u_i^2)} \cdot \sqrt{(\sum v_i^2)}}$$

Decision Boundary Calibration: A stored photograph P_k is identified as containing the guest if at least one facial embedding vector $\mathbf{v}_j$ satisfies:

$$\max_j S_C(\mathbf{u}, \mathbf{v}_j) \geq 0.45$$ (Optimized for ambient wedding lighting, angles, and facial accessories)

8. NON-FUNCTIONAL REQUIREMENTS (NFRS)

- **Performance:** Vector similarity search across 2,500 candidate faces executes in **≤ 250 milliseconds** using NumPy vectorized inner products.
- **Security:** Passwords protected with 12 Bcrypt salt rounds; JWT tokens signed via HMAC-SHA256 with environment secrets.
- **Guest Privacy Mandate:** Guest selfie images are ephemerally stored and destroyed via `cloudinary.uploader.destroy()` immediately post-inference.
- **Scalability:** Cloud-offloaded storage via Cloudinary CDN minimizes API gateway bandwidth and CPU load.

9. VERIFICATION, DIAGNOSTICS & TEST MATRIX

Test ID	Scenario / Feature	Input Conditions	Expected Output	Status
`TC-AUTH-01`	User Registration	Valid credentials & plan selection	User created with 50MB quota; returns 201	PASS
`TC-PKG-01`	Storage Quota Breached	Upload batch > available MB	Upload rejected; Cloudinary assets rolled back	PASS
`TC-EVT-01`	Cascade Event Deletion	Delete event with 20 photos	Purges Cloudinary assets, embeddings & photos	PASS
`TC-AI-01`	FaceNet Vector Extraction	Upload photo with 4 guest faces	4 `FaceEmbedding` JSON records created	PASS
`TC-AI-02`	Guest Selfie Face Match	WebRTC live camera selfie capture	Returns JSON list of matched photo records	PASS
`TC-AI-03`	Ephemeral Privacy Purge	Complete guest search request	Temporary selfie destroyed from Cloudinary	PASS
`TC-FLY-01`	Bilingual QR Flyer Canvas	Click "Download Printable QR Card"	Exports 800×1200 high-res PNG flyer	PASS

10. CONCLUSION & FUTURE ROADMAP

ShareEventPhoto AI delivers an innovative, production-grade cloud solution for intelligent event photograph distribution. Combining modern web frameworks (**Next.js 16, React 19, Express, Prisma, PostgreSQL**) with advanced deep learning computer vision (**DeepFace, FaceNet**), the platform achieves zero-friction guest photo discovery while preserving user privacy and providing flexible SaaS monetization for organizers.